OS PROJECT REVIEW | CO: 1 TO CO: 5 COMPLETE COVERAGE | C99 / POSIX

Mini-Container Runtime

A Lightweight Sandbox Virtualization Engine Built from Scratch in C Using Linux Namespaces & Unified Cgroups v2

Candidate: Tharakesha | **Course:** Operating Systems (CSE)
Kernel Subsystems: Namespaces, Cgroups v2, Demand Paging & VFS Sandboxing
Evaluation Tools: 5 Automated Test Suites + Real-time Interactive Menu

Concept: Containers vs Virtual Machines

Traditional Virtual Machines (VMs)

- Virtualize the **hardware layer** using hypervisors.
- Requires a full duplicate guest OS kernel per instance.
- High RAM footprint (GBs) and slow multi-second boot times.
- Incurs hardware emulation translation penalties.

OS-Level Containers (Our Implementation)

- Shares the single **underlying Linux kernel**.
- Containers are regular OS processes wrapped in kernel-enforced isolation boundaries.
- Sub-second boot times (10-20ms) and minimal RAM footprint.
- Near-native bare-metal execution performance.

Course Outcomes Alignment Matrix

SYLLABUS COVERAGE

Course Outcome (CO)	Topics Directly Addressed	Implementation Mechanism in C
CO 1: OS Principles & Processes	System calls, User/Kernel modes, PCB, Process creation	`clone()`, `execvp()`, `waitpid()`, custom child stack
CO 2: CPU Scheduling	CFS scheduling quotas, Proportional fair-share	Cgroups v2 `cpu.max` (quota/period) & `cpu.weight` (800 vs 200 contest)
CO 3: Concurrency & IPC	Synchronization primitives, Semaphores, IPC	POSIX `pipe()` barrier, Producer-Consumer with `sem_t` & shared memory
CO 4: Memory Management	Virtual memory, Demand paging, Page faults, OOM	Cgroups v2 `memory.max`, `pgfault` tracking (7,705 faults logged), OOM `SIGKILL`
CO 5: Virtualization & Filesystems	Container virtualization, Mount propagation, VFS	Namespaces (`PID`, `UTS`, `Mount`, `IPC`), `chroot()`, private `/proc`

System Architecture & Process Flow

PROCESS LIFECYCLE

Execution Flow Lifecycle

```
1. CLI Invocation:
   └─► sudo minicontainer run --memory 30M --cpu 40 /bin/sh

2. Host Parent Process:
   ├─► Allocates 1MB stack for child
   ├─► Opens UNIX synchronization pipe (barrier)
   ├─► clone(CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | ...)
   ├─► Creates /sys/fs/cgroup/minicontainer-[PID]
   ├─► Applies memory.max, cpu.max, pids.max
   ├─► Attaches child PID to cgroup.procs
   ├─► Unblocks child via sync pipe
   └─► Calls waitpid(), reads memory.stat, cleans cgroup

3. Container Child Process:
   ├─► Blocks on pipe until cgroups are attached
   ├─► sethostname("container-box")
   ├─► chroot() to rootfs & mounts private /proc
   └─► execvp(argv[0]) as PID 1
```

Key Engineering Highlights

- **Race-Free Attachment:** Child pauses on an IPC pipe until parent establishes cgroup limits, eliminating race conditions.
- **VFS Isolation:** Marks root mount as `MS_PRIVATE` to prevent container mounts from leaking into the host OS.
- **Real-Time Kernel Accounting:** Reads `memory.stat` upon exit to record peak RAM, minor page faults, and major page faults.

Core Isolation Primitives in Linux

KERNEL FEATURES

1. Namespaces

What the process can **SEE**

- `CLONE_NEWPID` : Child becomes PID 1; host processes hidden.
- `CLONE_NEWUTS` : Isolated hostname.
- `CLONE_NEWNS` : Isolated VFS mount table.
- `CLONE_NEWIPC` : Separate IPC semaphores & queues.

2. Cgroups v2

What the process can **USE**

- `memory.max` : Strict physical RAM ceiling; triggers OOM killer.
- `cpu.max` : CFS scheduler bandwidth quota.
- `cpu.weight` : Proportional fair share scheduling.
- `pids.max` : Prevents fork bomb attacks.

3. Rootfs Sandboxing

Where the process can **TOUCH**

- Uses Alpine Linux userland (~3.5MB).
- `chroot()` prevents traversal into host directories.
- Private `/proc` mount enables isolated `ps` output.

Empirical Results & Live Verification

LIVE VERIFICATION

1. Process Isolation (CO: 1 & CO: 5)

```
sudo ./minicontainer run --hostname sandbox
./tests/test_info
```

Result: Process is **PID 1**; parent PID 0; host processes hidden.

2. Memory Limit & OOM Killer (CO: 4)

```
sudo ./minicontainer run --memory 30M
./tests/test_memory 60
```

Result: Terminated by **SIGKILL (signal 9)**. Logged **7,705 Minor Page Faults** proving demand paging.

3. CPU Fair Share Contest (CO: 2)

```
sudo bash scripts/test_cpu_shares.sh
```

Result: Weight 800 vs 200 on Core 0: **2.78s vs 0.69s** (Exact 4:1 CPU execution ratio).

4. Semaphores & Shared Memory (CO: 3)

```
sudo ./minicontainer run ./tests/test_ipc_sync
```

Result: Producer-Consumer pipeline executed with zero race conditions via `sem_t`.

Technical Review & Viva Defense

FACULTY Q&A

Q1: Why clone() instead of fork()?

`fork()` duplicates the process in the exact same namespaces. `clone()` accepts flags like `CLONE_NEWPID` and `CLONE_NEWUTS`, creating the child directly inside brand-new, isolated kernel namespaces.

Q2: Why did memory limit trigger SIGKILL?

Linux uses optimistic memory allocation (overcommit) and demand paging. When `memset()` committed physical pages exceeding `memory.max` with swap disabled, the OS kernel OOM Killer dispatched an uncatchable `SIGKILL`.

Q3: Why is an IPC pipe needed?

To prevent a race condition. The child must not call `execvp()` before the parent finishes creating the cgroup and writing the child's PID into `cgroup.procs`.

Q4: How was CPU fair share verified?

Two containers with weights 800 and 200 ran concurrently on CPU 0. CFS allocated 2.78s and 0.69s of CPU time, verifying the theoretical 4:1 scheduling ratio with 99.5% accuracy.

EVALUATION DASHBOARD

Interactive Demonstration

Complete C runtime covering CO 1 to CO 5 with interactive menu and live kernel monitoring.

```
$ sudo bash interactive_menu.sh
```